Домашнее задание 3. Каталог: модель, структура и лайки

Продолжаем каталог из ДЗ 2: описываем модель данных, раскладываем код по файлам и оживляем лайки.

С этого задания каталог — одно приложение в папке lib, которое будет расти от задания к заданию до финального проекта.

Дедлайн: воскресенье, 23:59 по МСК.

1. Что нужно сделать

Основа — ваш каталог из ДЗ 2: та же тема и те же 8 и более карточек.

Образец макета (Figma, только просмотр): макет ДЗ 3 — экраны «ДЗ3 / Каталог: лайки и фильтр» и «ДЗ3 / Фильтр «Только избранные» включён». Как и раньше, дизайн можно менять под свой каталог.

Часть 1. Модель и структура проекта

1. Опишите модель (класс) одного элемента каталога: `id`, название, подпись, категория (`enum`, если в вашем каталоге есть категории), необязательное описание (`String?`). Поля `final`, конструктор `const`, обязательные поля — `required`. Можете добавить и другие поля по вашему желанию.
2. Разложите код по файлам:

```
lib/main.dart                          // запуск приложения
lib/domain/entities/<элемент>.dart     // модель и enum
lib/ui/widgets/<элемент>_card.dart     // карточка элемента
lib/ui/screens/catalog_screen.dart     // экран каталога
```

3. Список из 8 и более элементов объявите константой (`const`) в файле экрана, вне метода `build`.
4. Список карточек стройте из данных: `ListView.builder` (`itemBuilder`) или цикл `for-in` в `SingleChildScrollView`— без восьми вручную скопированных карточек. Карточка получает модель параметром.

5. Имена файлов — в snake_case, в каждом файле один главный класс. Свои файлы импортируйте относительным путём, например `import '../../domain/entities/book.dart';`.

Часть 2. Лайки

1. Экран каталога — `StatefulWidget`. Лайки храните в нём как набор id: `Set<int>`.
2. Карточка остаётся `StatelessWidget`: получает `isLiked` (лайкнута ли) и колбэк `onLikeTap`, который вызывает при нажатии на сердечко. Сама карточка `setState` не вызывает.
3. По нажатию на сердечко лайк ставится и снимается, экран перерисовывается через `setState`.
4. В шапке экрана — счётчик избранного, например «В избранном: 2».
5. Переключатель «Только избранные» (кнопка): когда он включён, в списке только лайкнутые элементы. Отфильтрованный список считайте в `build` через `where`, отдельно не храните. Если показывать нечего — текст «Ничего не нашлось».

Подсказки:

- Как карточка сообщает о нажатии: в карточке поле `final VoidCallback onLikeTap;`, на сердечке — `InkWell(onTap: onLikeTap, ...)` или `IconButton(onPressed: onLikeTap, ...)`. Функцию передаём без скобок.
- В экране карточку создаём так: `BookCard(book: book, isLiked: _likedIds.contains(book.id), onLikeTap: () => _toggleLike(book.id))`. Данные идут вниз параметрами, событие — вверх колбэком.
- Переключить лайк в `Set`: если id уже есть — `remove`, иначе `add`. Всё это внутри `setState`.
- Подчёркнуто красным `Undefined class` — значит, забыли импортировать нужный файл. Quick Fix (Cmd+. на Mac, Ctrl+. на Windows) добавит импорт сам.

Запустите проект и проверьте: лайки ставятся и снимаются, счётчик меняется, фильтр показывает только избранное. После этого переходите к сдаче.

2. Как сдать работу через GitHub

Работаем в том же репозитории и проекте. С этого задания каталог живёт прямо в `lib/`, а не в папке `hwN`. Папки `lib/hw1` и `lib/hw2` не трогайте — пусть остаются как архив, они не мешают.

Шаг 1. Смерджите hw2 (в этот раз обязательно)

Откройте Pull Request по ДЗ 2 на GitHub, нажмите зелёную кнопку Merge pull request, затем Confirm merge. В этот раз это нужно: иначе в новой ветке не будет кода из `lib/hw2`, который мы переносим. Смерженный Pull Request по-прежнему открывается по ссылке, проверка от этого не пострадает.

Шаг 2. Обновите main

Откройте терминал в корне проекта (там, где лежит pubspec.yaml) и выполните:

```
git checkout main
git pull origin main
```

Первая команда переключает вас на ветку main, вторая подтягивает её последнюю версию с GitHub — уже вместе со смердженным ДЗ 2.

Шаг 3. Создайте ветку hw3

```
git checkout -b hw3
```

Команда создаёт ветку hw3 и сразу переключает вас на неё.

Шаг 4. Создайте новую структуру папок

В папке lib создайте папки `domain/entities`, `ui/screens` и `ui/widgets`. Должно получиться так:

```
lib/
├── main.dart
├── domain/entities/
├── ui/screens/
├── ui/widgets/
```

```
├── hw1/          (не трогаем)
└── hw2/          (не трогаем)
```

Шаг 5. Перенесите код из ДЗ 2

Код из `lib/hw2/main.dart` не удаляйте, а скопируйте по частям: вёрстку карточки — в `ui/widgets/<элемент>_card.dart`, экран со списком — в `ui/screens/catalog_screen.dart`.

Файл `lib/main.dart` (сейчас там стандартный счётчик Flutter) замените полностью: в нём остаются только `main()` и `MaterialApp` с `home: const CatalogScreen()`.

Шаг 6. Реализуйте задание

Опишите модель, подключите её к карточке и экрану, сделайте лайки и фильтр. Запуск теперь обычный, без указания файла:

```
flutter run
```

Если захотите посмотреть старое ДЗ, укажите его файл: `flutter run -t lib/hw2/main.dart`.

Шаг 7. Закоммитьте и отправьте на GitHub

```
git add .
git commit -m "hw3: model, structure, likes"
git push -u origin hw3
```

После этого на GitHub появится ветка hw3 с вашим решением.

Шаг 8. Откройте Pull Request в main

Зайдите на страницу репозитория на GitHub. Обычно сразу видна жёлтая плашка «hw3 had recent pushes» с кнопкой Compare & pull request, нажмите её. Если плашки нет, откройте вкладку Pull requests → New pull request, слева укажите base: main, справа compare: hw3.

Проверьте, что сравниваются именно main и hw3, добавьте короткое название (например «hw3») и нажмите Create pull request. **Сами этот Pull Request не мерджите: он должен остаться открытым.**

Шаг 9. Запишите демо

Запишите видео экрана на 10–20 секунд: поставьте и снимите пару лайков так, чтобы было видно, как меняется счётчик, затем включите и выключите «Только избранные».

Шаг 10. Сдайте работу в LMS

До дедлайна загрузите на платформу:

1. видео с демо: лайки, счётчик и фильтр;
2. ссылку на открытый Pull Request. Скопируйте адрес страницы Pull Request из адресной строки браузера, он выглядит как https://github.com/ваш-логин/название/pull/3.

3. Важно

- Репозиторий обязательно должен быть публичным (Public), иначе я не смогу открыть ваш код.
- Pull Request по hw3 должен быть открыт к моменту проверки, не мерджите его сами.
- Не удаляйте и не переименовывайте ветку main, вся работа идёт в отдельных ветках вроде hw3.
- Если на любом шаге команда выдаёт ошибку, не удаляйте и не пересоздавайте всё с нуля, пришлите мне текст ошибки, разберёмся.